ELECTRÓNICA DIGITAL I · SEGUNDA ENTREGA · CLASE 4 DE 5

Máquinas de estado

fsm_proceso: la máquina que decide cuándo se enciende la bomba y cuándo se abre la compuerta.

DOS FAMILIAS DE MÁQUINAS

En el proyecto, los actuadores son Moore

	Moore	Mealy
Salida depende de	solo el estado	estado y entradas
Reacciona	un ciclo después	en el mismo ciclo
Glitches	no, si sale de un registro	sí, si la entrada cambia
En el proyecto	bomba_en, compuerta_en (R6)	comandos internos entre bloques

Un ciclo de retraso son 20 ns. Para una bomba, eso no importa; un pulso falso de 5 ns que la encienda, sí.

MÉTODO · R5

Del diagrama al código en tres traducciones

1

Círculos → tipo

```
type t_estado is
(REPOSO, LLENAR,
 VACIAR, ...);
```

Cada estado del diagrama es un valor del tipo enumerado.

2

Flechas → P2

```
case estado is
 when REPOSO =>
  if ... then
```

Cada flecha es un if dentro del when de su estado de origen.

3

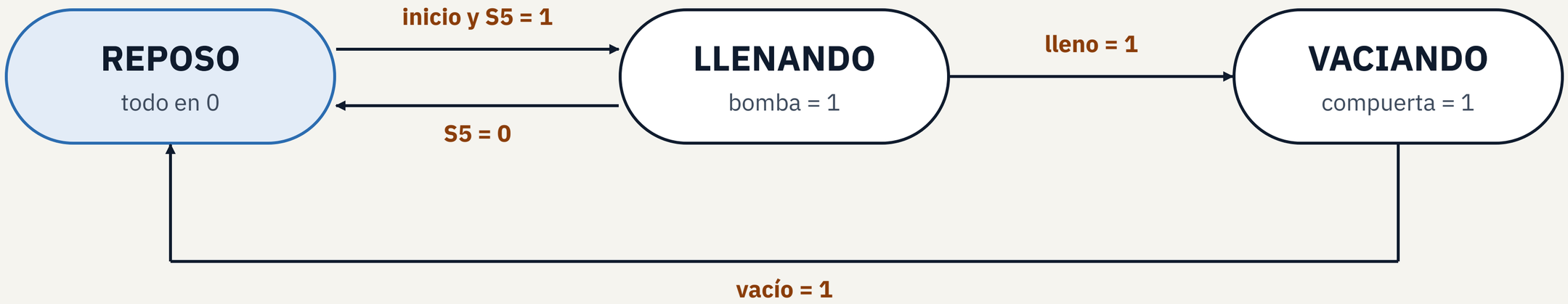
Rótulos → P3

```
case estado_n is
 when LLENAR =>
  bomba_n <= '1';
```

Lo escrito dentro de cada círculo es una salida.

CALENTAMIENTO · ANALIZAR/FSM_TANQUE_SIMPLE.VHD

Un tanque simplificado con tres estados



El archivo trae esta misma máquina escrita con un proceso y con tres. Compárenlas: ¿cuál se lee directo del diagrama?

ESTRUCTURA DE FSM_PROCESO

Tres procesos, tres responsabilidades

P1 · registros

```
if rising_edge(clk)
 if rst = '1' then
  estado <= REPOSO;
 else
  estado <= estado_n;
  bomba_r <= bomba_n;
  comp_r  <= comp_n;
 end if;
end if;
```

P2 · siguiente estado

```
estado_n <= estado;
case estado is
 when REPOSO =>
  if cmd_nivel = '1'
   and s5 = '1' ...
   estado_n <=
    LLENAR_NIVEL;
  end if;
 ...
```

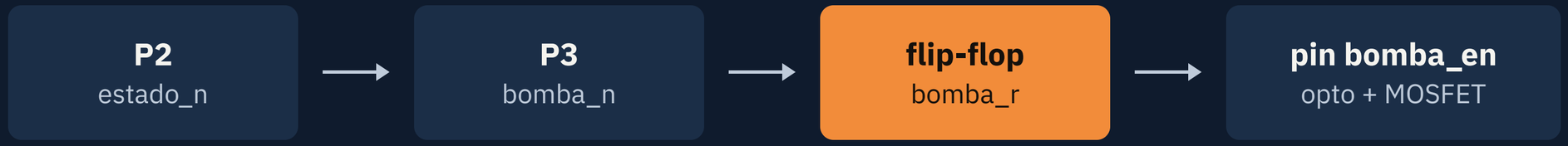
P3 · salidas

```
bomba_n <= '0';
comp_n  <= '0';
case estado_n is
 when LLENAR_NIVEL
  | LLENAR_TIEMPO =>
  bomba_n <= '1';
 ...
```

P2 y P3 empiezan con valores por defecto: así ningún camino deja una señal sin asignar y no aparecen latches.

SALIDAS REGISTRADAS · R6

Lo que toca el mundo físico sale de un flip-flop



Mientras estado cambia, sus bits no cambian todos a la vez. Una salida combinacional puede dar un pulso falso de nanosegundos.

El flip-flop solo copia en el flanco, cuando todo ya se estabilizó. El pin nunca ve ese pulso.

TABLA DE TRANSICIONES DE FSM_PROCESO · FIGURA 2 DE LA GUÍA

Media tabla hecha, media tabla de taller

Desde	Condición (en orden de prioridad)	Hacia
todos menos FALLA	error de sensor o watchdog	FALLA
REPOSO	cmd_nivel, S5 = 1 y nivel < objetivo	LLENAR_NIVEL
REPOSO	cmd_tiempo y S5 = 1	LLENAR_TIEMPO · t_cargar
REPOSO	pedido manual	MANUAL
LLENAR_*	nivel = 100 % → S5 = 0 → cancelar → objetivo o t = 0	VACIAR → PAUSA → REPOSO
PAUSA	**TODO 1**	VACIAR, REPOSO u origen
VACIAR	**TODO 2**	REPOSO
MANUAL	**TODO 3**	VACIAR o REPOSO
FALLA	**TODO 4**	REPOSO

PRIORIDADES · SECCIÓN 5.1 DE LA GUÍA

El orden de los elsif es la prioridad

```
-- prioridad 1, antes del case:
if estado /= FALLA and hay_falla = '1' then
  estado_n <= FALLA;
else case estado is
  when LLENAR_NIVEL | LLENAR_TIEMPO =>
    if lleno = '1' then -- 2
      estado_n <= VACIAR;
    elsif s5 = '0' then -- 3
      estado_n <= PAUSA;
    elsif cmd_cancelar = '1' then -- 4
      estado_n <= REPOSO;
```

Caso límite

El nivel llega a 100 % justo cuando el usuario presiona C.

Gana VACIAR, porque su if va primero. Si invirtieran el orden, el tanque lleno quedaría con la compuerta cerrada.

MEMORIA DENTRO DE LA FSM

Dos estados que necesitan recordar algo más

PAUSA

Se llega desde LLENAR_NIVEL o desde LLENAR_TIEMPO cuando S5 = 0. Al volver el agua, ¿a cuál regresa?

Un registro **origen** se carga en la misma transición: origen_n <= estado.

VACIAR

Se entra al llegar a 100 %, pero en cuanto baja a 75 % la condición ya no se cumple.

El **estado** es el enclavamiento: solo sale cuando nivel = 0 %.

INTERBLOQUEO BOMBA–COMPUERTA · RF6

Imposible por construcción, vigilado en cada ciclo

En el diseño · P3, estado MANUAL

```
comp_n  <= man_comp;
bomba_n <= man_bomba
   and s5
   and not man_comp;
```

Si piden las dos, gana la compuerta. No hay combinación de entradas que encienda ambas.

En el testbench · siempre activo

```
if rising_edge(clk) then
  assert not (bomba = '1'
    and comp = '1')
  severity failure;
```

Si alguna vez ocurre, la simulación se detiene en el instante exacto.

TALLER EN CLASE

Primero la tabla en papel, después el código

1 · Papel
Completen las 4 filas naranja de la tabla de transiciones.

2 · TODO 1 a 4
PAUSA, VACIAR, MANUAL y FALLA en P2.

3 · TODO 5 y 6
Salidas de VACIAR y MANUAL en P3.

22 OK tb_fsm_proceso prueba cada escenario por separado. Con la plantilla sin tocar: 9 OK y 13 FALLAS.

Tarea: diagrama de estados de fsm_sesion en papel, con los estados mínimos de la sección 5.2. Se usa en la clase 5.